

Optimisation: exhaustive search versus decision variable elimination

Simon Frost

Table of contents

1	Overview	1
2	Setup	2
3	Exhaustive policy search	2
4	The valuation algebra	3
5	The strong elimination order	4
6	Agreement and timing	5
7	Summary	6
8	References	6

1 Overview

`optimize` solves $\sigma^* = \arg \max_{\sigma} EU(\sigma)$ (SPEC section 32) with one of two backends:

- `ExhaustivePolicySearch` enumerates every deterministic strategy; it is the correctness oracle of SPEC section 55.6;
- `DecisionVariableElimination` generalises variable elimination with a valuation algebra that sums out chance variables and maximises out decisions (SPEC section 33). This is dynamic programming over the diagram (Tatman and Shachter 1990) carried out on the division algebra of Jensen et al. (1994).

Both return a `DecisionSolution` with the maximal expected utility, an optimal strategy of deterministic policies, and diagnostics.

2 Setup

```
using InfluenceDiagrams
using BayesianNetworkInference: unit_factor, reorder
m = umbrella_model()
```

```
InfluenceDiagramModel(3 variables, 1 decision, 1 utility, 2 kernels, 1 bound)
```

3 Exhaustive policy search

The number of deterministic strategies is the product over decisions of $|A_D|^{|I_D|}$; the umbrella decision has $2^3 = 8$.

```
n_deterministic_policies(m, :Umbrella)
```

```
8
```

```
ex = optimize(m, ExhaustivePolicySearch())
```

```
DecisionSolution(expected_utility = 77.0, Strategy(Umbrella => DeterministicPolicy))
```

```
ex.expected_utility, policy_table(ex.strategy[:Umbrella]), ex.diagnostics
```

```
(77.0, [:leave, :leave, :take], (nstrategies = 8, method = :value_table))
```

By default each strategy is scored from a value table built once by enumerating all joint states with the actions left free; `brute_force = true` instead evaluates every strategy through `instantiate` and `expected_utility`, the slowest and most literal form of the oracle. The search refuses more than `max_policies` strategies.

```
optimize(m, ExhaustivePolicySearch(; brute_force = true)).expected_utility
```

```
76.99999999999999
```

4 The valuation algebra

Decision variable elimination works on pairs (ϕ, ψ) of factors: a probability potential and a utility potential in “already divided” form, together denoting the contribution $\phi \cdot \psi$ to the expected utility (the division algebra of Jensen et al. (1994)). The operations are

- combination: $(\phi_1, \psi_1) \otimes (\phi_2, \psi_2) = (\phi_1 \phi_2, \psi_1 + \psi_2)$;
- sum-elimination of a chance variable X : $\phi' = \sum_X \phi$, $\psi' = \sum_X \phi \psi / \phi'$ with $0/0 := 0$;
- max-elimination of a decision D : $\psi' = \max_D \psi$, with ϕ constant in D (an error otherwise) and the maximising action recorded as a policy table.

A set of valuations denotes $(\prod_i \phi_i)(\sum_i \psi_i)$, so eliminating a variable only combines the valuations whose scope contains it. The initial valuations are $(\kappa_X, 0)$ per chance mechanism and $(1, u_j)$ per utility; at the end ϕ is the probability of the evidence and ψ the maximal expected utility.

```
W = FiniteAxis(:Weather, [:sunny, :rainy]); F = FiniteAxis(:Forecast, [:sunny,
:cloudy, :rainy])
U = FiniteAxis(:Umbrella, [:take, :leave])
pw = Valuation(Factor([W], [0.7, 0.3]))
pf = Valuation(Factor([W, F], [0.7 0.2 0.1; 0.15 0.25 0.6]))
u = Valuation(unit_factor(), Factor([W, U], [20.0 100.0; 70.0 0.0]))
v = combine([pw, pf, u])
```

Valuation{Float64} over (:Weather, :Forecast, :Umbrella) (ϕ over (:Weather, :Forecast), ψ over (:Weather, :Umbrella))

Summing out the weather divides by the marginal of the forecast, so ψ becomes the conditional expected utility $E[U \mid F, D]$:

```
s = sum_out(v, :Weather)
reorder(s.ψ, [:Forecast, :Umbrella]).table
```

```
3×2 Matrix{Float64}:
 24.2056  91.5888
 37.4419  65.1163
 56.0     28.0
```

Maximising out the umbrella records the argmax for every forecast and leaves $E[\max_D E[U \mid F, D]]$ to be summed over the forecast:

```
r, policy, policy_scope = max_out(s, :Umbrella)
policy, policy_scope, sum_out(r, :Forecast).ψ.table[]
```

```
([:leave, :leave, :take], [:Forecast], 77.00000000000001)
```

5 The strong elimination order

Chance variables must be summed out and decisions maximised in an order compatible with what the decision maker knows: the variables are partitioned into blocks $I_0 < D_1 < I_1 < \dots < D_n < I_n$, where I_{k-1} holds the chance variables first observed at D_k and I_n the ones never observed, and elimination runs from the last block backwards with a min-fill order inside each chance block (BayesianNetworkInference.elimination_order on the current interaction graph).

```
strong_elimination_order(m)
```

```
3-element Vector{Symbol}:
 :Weather
 :Umbrella
 :Forecast
```

```
dve = optimize(m, DecisionVariableElimination())
```

```
DecisionSolution(expected_utility = 77.00000000000001, Strategy(Umbrella =>
DeterministicPolicy))
```

```
dve.expected_utility, policy_table(dve.strategy[:Umbrella]), dve.diagnostics
```

```
(77.00000000000001, [:leave, :leave, :take], (order = [:Weather, :Umbrella,
:Forecast], max_factor_size = 6, policy_scopes = Dict{:Umbrella => [:Forecast]}),
evidence_probability = 0.9999999999999999))
```

The algorithm is exact when, at each maximisation, the utility potential depends only on variables the decision maker observes: the *no-forgetting* (perfect recall) subset, in which everything known at a decision is still known at every later one. (Shachter (1986) reserves *regular* for the weaker property that a directed path runs through all the decisions.) No-forgetting is not a validity condition – a diagram that forgets is a well-formed limited-memory diagram (Lauritzen and Nilsson 2001), which ExhaustivePolicySearch solves exactly – so it is checked at run time instead: the two-stage test-then-act problem satisfies it, and the same diagram with Drill forgetting Test does not.

```
t = two_stage_model()
sol = optimize(t)
sol.expected_utility, sol.strategy[:Test](), policy_table(sol.strategy[:Drill])
```

```
(21.0, :test, [:drill :dont :drill; :drill :drill :drill])
```

```
try
    optimize(without_information(t, :Drill, :Test))
catch e
    println(sprint(showerror, e))
end
```

IrregularDiagramError: when decision :Drill is maximised, the utility potential still depends on Test, which the decision maker does not observe; decision variable elimination is exact only for diagrams with no-forgetting (perfect recall). Add the missing information arcs with with_no_forgetting, or solve this limited-memory diagram with ExhaustivePolicySearch

The error names the variables the decision maker would have to remember and the two ways out: no_forgetting_arcs lists the missing arcs, with_no_forgetting adds them, and ExhaustivePolicySearch solves the diagram as it stands. Vignette 04 works through the repair.

```
forgetful = without_information(t, :Drill, :Test)
(missing_arcs = no_forgetting_arcs(forgetful),
 repaired = optimize(with_no_forgetting(forgetful)).expected_utility,
 limited_memory = optimize(forgetful, ExhaustivePolicySearch()).expected_utility)
```

```
(missing_arcs = [:Drill => :Test], repaired = 21.0, limited_memory = 21.0)
```

6 Agreement and timing

SPEC section 55.6 requires the two backends to agree on expected utility and optimal policy; the test suite checks this on the umbrella, the SPEC section 46 grazing diagram, the two-stage problem and random tiny diagrams. On the grazing diagram exhaustive search enumerates $3^9 = 19683$ strategies.

```
g = reference_grazing_model()
optimize(g, ExhaustivePolicySearch()); optimize(g); # compile both backends first
t_ex = @elapsed sol_ex = optimize(g, ExhaustivePolicySearch())
t_dve = @elapsed sol_dve = optimize(g)
(exhaustive = sol_ex.expected_utility, dve = sol_dve.expected_utility,
 seconds_exhaustive = round(t_ex; digits = 3), seconds_dve = round(t_dve; digits =
3))
```

```
(exhaustive = 36.52414686249998, dve = 36.524146862500004, seconds_exhaustive =
0.283, seconds_dve = 0.001)
```

```
(julia = string(VERSION), cpu = Sys.CPU_NAME, threads = Threads.nthreads(),
 repetitions = 1, statistic = "single run after warm-up")
```

```
(julia = "1.12.7", cpu = "apple-m1", threads = 1, repetitions = 1, statistic =
"single run after warm-up")
```

These times describe this execution, not a portable speedup guarantee. Agreement here is numerical evidence on this example; exact-model theorems do not imply bit-for-bit agreement for arbitrary floating-point utilities.

```
policy_table(sol_ex.strategy[:GrazingManagement]) ==  
policy_table(sol_dve.strategy[:GrazingManagement])
```

true

Ties between actions are broken in favour of the first action label by both backends (argmax_table for elimination, enumeration order for the search), so the recovered tables also agree whenever the optimum is unique on every reachable information state. The elimination diagnostics record the order used, the largest potential and the variables each policy really depends on:

```
sol_dve.diagnostics
```

```
(order = [:Biodiversity, :GrazingPressure, :Occupancy, :HabitatQuality, :Vegetation,  
:SoilMoisture, :Climate, :GrazingManagement, :CurrentVegetation, :ClimateForecast],  
max_factor_size = 162, policy_scopes = Dict(:GrazingManagement => [:ClimateForecast,  
:CurrentVegetation]), evidence_probability = 0.9999999999999999)
```

7 Summary

In exact arithmetic on the supported model class, both backends return the same maximal expected utility and, where the optimum is unique on every reachable information state, the same policy tables: exhaustive search is the oracle, and decision variable elimination is the algorithm that scales, eliminating in a strong order that sums out chance variables and maximises out decisions on the valuation algebra. Elimination is exact only under no-forgetting, which is checked at run time rather than by validate; external evidence must also have no action among its causal ancestors. Unsupported cases require explicitly choosing the exhaustive oracle rather than an automatic fallback. The next vignette, *Value of information*, uses the same two optimisations twice over to price an observation.

8 References

- Jensen, Frank, Finn V. Jensen, and Søren L. Dittmer. 1994. “From Influence Diagrams to Junction Trees.” *Proceedings of the Tenth Conference on Uncertainty in Artificial Intelligence (UAI 1994)*, 367–73.
- Lauritzen, Steffen L., and Dennis Nilsson. 2001. “Representing and Solving Decision Problems with Limited Information.” *Management Science* 47 (9): 1235–51. <https://doi.org/10.1287/mnsc.47.9.1235.9779>.
- Shachter, Ross D. 1986. “Evaluating Influence Diagrams.” *Operations Research* 34 (6): 871–82. <https://doi.org/10.1287/opre.34.6.871>.

Tatman, Joseph A., and Ross D. Shachter. 1990. "Dynamic Programming and Influence Diagrams." *IEEE Transactions on Systems, Man, and Cybernetics* 20 (2): 365–79. <https://doi.org/10.1109/21.52545>.